Open in app ↗

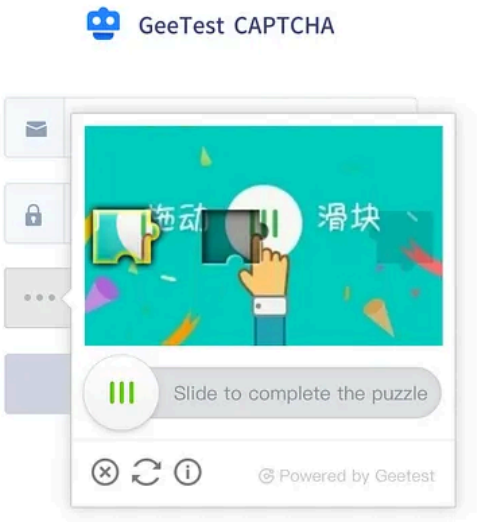
Medium Search Write

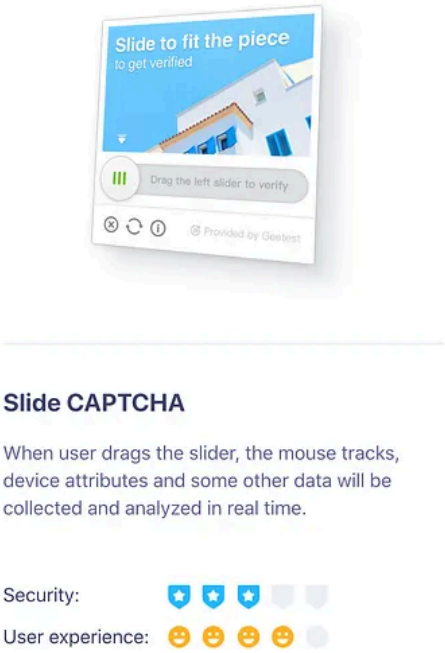
Get unlimited access to the best of Medium for less than $1/week. Become a member

How to solve GeeTest “slider CAPTCHA” with JS

Filip Vitas Follow 4 min read · Jun 8, 2020

377 7





> *My previous story on this topic was:*
> *How to bypass “slider CAPTCHA” with JS and Puppeteer*

In this story, I will go further to solve slider captcha with a different technique. This technique will solve the slider captcha faster and more efficiently. My focus will be on GeeTest slider captcha, but you could apply it to any other similar slider captcha. I will show you how to bypass it in a few steps.

1. Get images

Let’s go to the GeeTest website. Our puppeteer script needs to wait a bit for images to load. When everything is loaded, the script will get the images from canvases.

```
const puppeteer = require('puppeteer')
const fs = require('fs').promises

async function run () {
    const browser = await puppeteer.launch({
        headless: true,
        defaultViewport: { width: 1366, height: 768 }
    })
    const page = await browser.newPage()

    await page.goto('https://www.geetest.com/en/demo', { waitUntil: 'networkidle2' })

    await page.waitFor(3000)

    await page.waitForSelector('.tab-item.tab-item-1')
    await page.click('.tab-item.tab-item-1')

    await page.waitForSelector('[aria-label="Click to verify"]')
    await page.waitFor(1000)

    await page.click('[aria-label="Click to verify"]')

    await page.waitForSelector('.geetest_canvas_img canvas', { visible: true })
    await page.waitFor(1000)
    let images = await page.$$eval('.geetest_canvas_img canvas', canvases => {
        return canvases.map(canvas => canvas.toDataURL().replace(/^data:image\/png; base64,/, ''))
    })

    await fs.writeFile(`./captcha.png`, images[0], 'base64')
    await fs.writeFile(`./puzzle.png`, images[1], 'base64')
    await fs.writeFile(`./original.png`, images[2], 'base64')

    await browser.close()
}

run()
```

With this code, we get the original image, captcha image and puzzle image. That’s all we need to make a diff and to know exactly where to move the slider.

Original image (left) vs captcha image (right)

On the captcha image, there is a subtle shadow puzzle piece. That shadow piece should have some protective purpose. It has no purpose at all. It’s easy to filter it out in our next step.

2. Picture diffing

There are few great js libraries for image processing and diffing.
For diffing I used pixelmatch.

```
const Jimp = require('jimp')
const pixelmatch = require('pixelmatch')

async function run() {
    const originalImage = await Jimp.read('./original.png')
    const captchaImage = await Jimp.read('./captcha.png')

    const { width, height } = originalImage.bitmap
    const diffImage = new Jimp(width, height)

    const diffOptions = { includeAA: true, threshold: 0.2 }

    pixelmatch(originalImage.bitmap.data, captchaImage.bitmap.data, diffImage.bitmap.data, width, height, diffOptions)
    diffImage.write('./diff.png')
}

run()
```

After we run diffing code, we will get the diff image that looks like this:

3. Locate the diff

Now that we have the diff image, we need to locate the x coordinate of the puzzle whole in the diff image. In this step, I will use OpenCV js library for image processing. We have a few options:

- Browser version: opencv.js
- Nodejs bindings for native OpenCV: opencv4nodejs

- Precompiled OpenCV to wasm for Nodejs: opencv-wasm

Since I’m running code in Node and I don’t want to install and compile OpenCV, I decided to use opencv-wasm.

We need to translate that diff image into something better. So let’s apply ***threshold*** to eliminate all noise, ***erode*** to fill all white gaps and ***dilate*** to revert the effects of erosion after filling all the gaps in the image.

```
let srcImage = await Jimp.read('./diff.png')
let src = cv.matFromImageData(srcImage.bitmap)

let dst = new cv.Mat()
let kernel = cv.Mat.ones(5, 5, cv.CV_8UC1)
let anchor = new cv.Point(-1, -1)

cv.threshold(src, dst, 127, 255, cv.THRESH_BINARY)
cv.erode(dst, dst, kernel, anchor, 1)
cv.dilate(dst, dst, kernel, anchor, 1)
```

Now it’s time to find the center of that puzzle whole.

```
let srcImage = await Jimp.read('./diff.png')
let src = cv.matFromImageData(srcImage.bitmap)
let dst = new cv.Mat()

cv.cvtColor(src, src, cv.COLOR_BGR2GRAY)
cv.threshold(src, dst, 150, 255, cv.THRESH_BINARY_INV)

let contours = new cv.MatVector()
let hierarchy = new cv.Mat()
cv.findContours(dst, contours, hierarchy, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)

let contour = contours.get(0)
let moment = cv.moments(contour)
let cx = Math.floor(moment.m10 / moment.m00)
let cy = Math.floor(moment.m01 / moment.m00)

// cx is what we need
console.log(cx, cy)

cv.cvtColor(dst, dst, cv.COLOR_GRAY2BGR)
let redColor = new cv.Scalar(255, 0, 0)

cv.drawContours(dst, contours, 0, redColor)
cv.circle(dst, new cv.Point(cx, cy), 3, redColor)
cv.putText(dst, 'center', new cv.Point(cx + 4, cy + 3), cv.FONT_HERSHEY_SIMPLEX, 0.5, redColor)

new Jimp({ width: dst.cols, height: dst.rows, data: Buffer.from(dst.data) }).write('./diff.png')
```

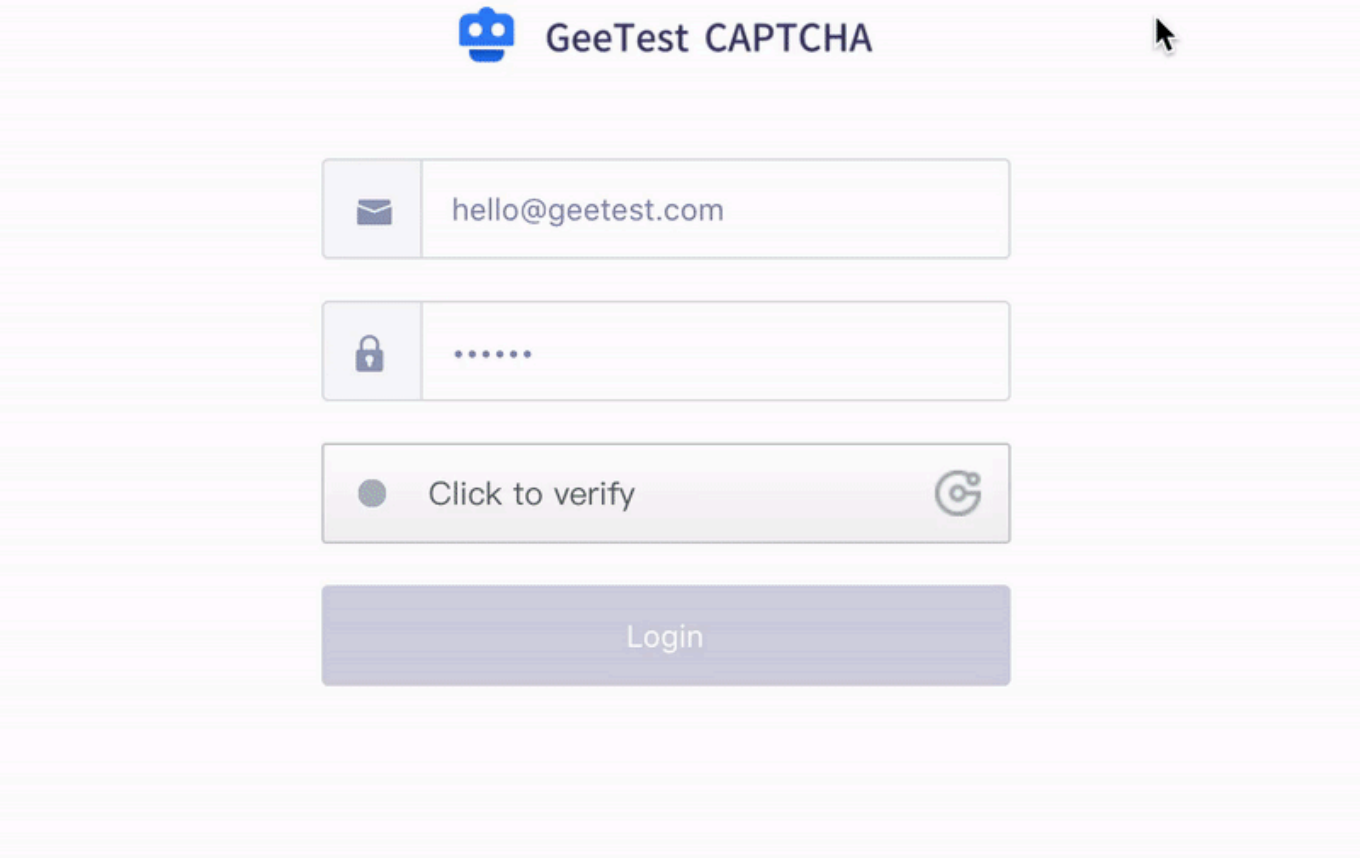
We found the position where we need to move the puzzle piece.

4. Move slider to the position

Moving slider is not easy as it seems. We have one more challenge. Somewhere at the beginning, puzzle jumps with some random offset. It means that the slider position and the puzzle position are not in sync.

We need to move slider 2 times. The first move will bring the puzzle closer to the final position. Then we calculate the puzzle position and how much we need to move it again. The second move will bring the puzzle exactly where it needs to be in order to solve GeeTest slider captcha.

```
const browser = await puppeteer.launch({
    headless: false,
    defaultViewport: { width: 1366, height: 768 }
})
const page = await browser.newPage()

await page.goto('https://www.geetest.com/en/demo', { waitUntil: 'networkidle2' })

await page.waitFor(1000)

await saveSliderCaptchaImages(page)
await saveDiffImage()

let [cx, cy] = await findDiffPosition(page)

const sliderHandle = await page.$('.geetest_slider_button')
const handle = await sliderHandle.boundingBox()

let xPosition = handle.x + handle.width / 2
let yPosition = handle.y + handle.height / 2
await page.mouse.move(xPosition, yPosition)
await page.mouse.down()

xPosition = handle.x + cx - handle.width / 2
yPosition = handle.y + handle.height / 3
await page.mouse.move(xPosition, yPosition, { steps: 25 })

await page.waitFor(100)

let [cxPuzzle, cyPuzzle] = await findPuzzlePosition(page)

xPosition = xPosition + cx - cxPuzzle
yPosition = handle.y + handle.height / 2
await page.mouse.move(xPosition, yPosition, { steps: 5 })
await page.mouse.up()

// success!

await browser.close()
```



All code for solving GeeTest “slider CAPTCHA” is uploaded to GitHub repo. Feel free to copy anything you like. This is for educational purpose, use Puppeteer responsibly and have fun.
If you try to solve captcha too many times it may stop working.

Conclusion

GeeTest will eventually figure out how to make this slider CAPTCHA harder to solve or they will drop this silly slider CAPTCHA because it’s not secure at all.

. . .

Thanks for reading! If you like the article, give a clap, or two, or 50 👏.
Leave a comment below if you have any questions or say hi on Twitter.

JavaScript Bots Js Web Web Development

Written by Filip Vitas

252 followers · 17 following

Coffee Driven Software Developer @Everseen

Follow

Responses (7)

Femi Aws Storage

What are your thoughts?

See all responses